



Concurrency: An Introduction

Week 10: Threads, Race Conditions & Critical Sections

lectura.id/course/os

Program Studi Teknik Informatika
Institut Teknologi Sumatera

2026

OUTLINE

Peta materi pertemuan ini

1. **Pembuka & Motivasi**
2. **Thread Abstraction**
3. **Race Condition**
4. **Critical Section**
5. **Ringkasan & Penutup**

Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Menjelaskan apa itu thread dan bagaimana thread berbeda dari proses.
2. Mengidentifikasi sumber daya apa yang di-*share* dan apa yang bersifat private pada thread.
3. Membaca kode C sederhana dengan thread dan memprediksi kemungkinan outputnya.
4. Mendefinisikan *race condition* dan memberikan contoh konkretnya.
5. Menjelaskan mengapa `counter++` bisa menghasilkan hasil salah ketika dijalankan oleh dua thread.
6. Mendefinisikan *critical section* beserta tiga syarat solusinya.

Fokus Kelas Ini

Kita fokus pada **perbedaan thread dan proses**, **race condition**, dan **critical section** secara step-by-step.

Pembuka & Motivasi

Mengapa Kita Perlu Concurrency?

Dunia nyata tidak sekuensial

Aplikasi nyata yang perlu concurrency:

- **Browser:** Download file + render halaman + handle input user → **bersamaan**
- **Server web:** Ratusan request masuk secara bersamaan
- **Video game:** Render grafis + update AI + handle input → paralel
- **Smartphone:** Background service + UI responsiveness

Bayangkan...

Jika browser tidak bisa download sambil merender, maka:

- Download selesai → baru render
- User freeze saat download
- Aplikasi terasa "beku"

Sangat jelek untuk UX!

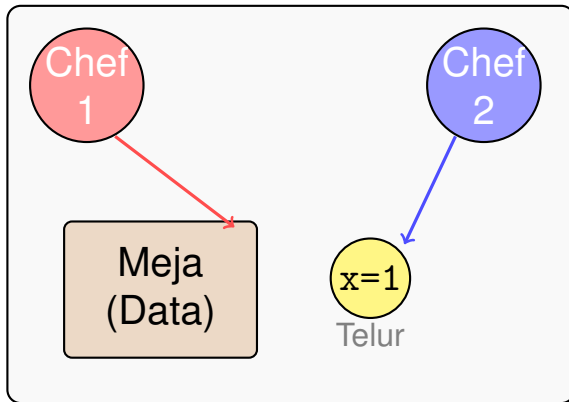
Tantangan

Concurrency bawa masalah baru: jika dua thread mengakses data yang sama, hasil bisa tidak terduga.

Analogi: Dua Chef di Dapur yang Sama

Bayangkan dua chef berbagi satu dapur...

Dapur (Shared Memory)



*Apa yang terjadi jika keduanya mengambil telur di **waktu bersamaan**?*

Analogi: Dua Chef — Pembahasan

Memetakan analogi ke konsep OS

Skenario Masalah

- **Chef 1** ambil telur terakhir
- **Chef 2** juga ambil telur di waktu bersamaan
- Hasil: telur hilang — **data rusak!**

Analoginya

- Chef → **thread**
- Dapur → **shared memory**
- Telur → **shared variable**

Solusi: Butuh Koordinasi

Perlu mekanisme **antrian** atau **zona eksklusif** agar tidak tabrakan — inilah yang akan kita bahas!

Agenda Kelas Hari Ini

Apa yang kita akan pelajari hari ini

Bagian 1: Thread Abstraction

Apa itu thread? Bagaimana thread berbeda dari proses? Apa yang *shared* dan apa yang *private*?

Bagian 2: Race Condition

Apa itu *race condition*? Kenapa `counter++` bisa menghasilkan nilai salah?

Bagian 3: Critical Section

Apa itu *critical section*? Apa tiga syarat solusinya?

Bagian 4: Ringkasan & Latihan

Review konsep, latihan soal, dan Q&A.

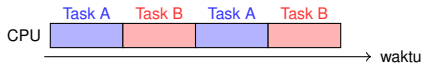
Concurrency vs Parallelism

Dua konsep yang sering tertukar

Concurrency

Satu CPU, banyak task bergantian dengan cepat.

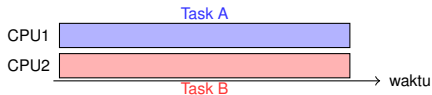
Terasa berjalan bersamaan, tapi sebenarnya **switching cepat**.



Satu koki bergantian aduk sup dan potong sayur

Parallelism

Banyak CPU, banyak task berjalan **benar-benar bersamaan**.



Dua koki masing-masing mengerjakan tugasnya sendiri

Checkpoint 0

Cek pemahaman sebelum lanjut

1. Apa perbedaan utama antara **concurrency** dan **parallelism**?
2. Mengapa browser perlu concurrency?
3. Dalam analogi dapur, apa representasi dari *thread* dan *shared memory*?

Kunci Jawaban Singkat

1. Concurrency = satu CPU switching antar task (ilusi serentak); Parallelism = banyak CPU berjalan benar-benar bersamaan.
2. Agar download, render, dan input handling bisa terjadi tanpa user freeze.
3. Chef = thread; Dapur = shared memory; Telur = shared variable.

Thread Abstraction

Thread vs Process

Perbedaan dan keuntungan thread

Masalah dengan Process murni:

- **Fork** membuat **copy penuh** address space
→ mahal
- Komunikasi antar proses (IPC) kompleks:
pipe, socket
- Context switch **berat**: harus ganti page table

Thread sebagai solusi:

- Thread = unit eksekusi **di dalam** proses
- Berbagi address space yang sama
- Lebih ringan: buat dan switch lebih cepat

Aspek	Process	Thread
Address space	Masing-masing	Shared
Buat over-head	Besar	Kecil
Context switch	Berat	Ringan
Komunikasi	IPC kompleks	Shared mem
Crash satu	Isolasi	Crash proses

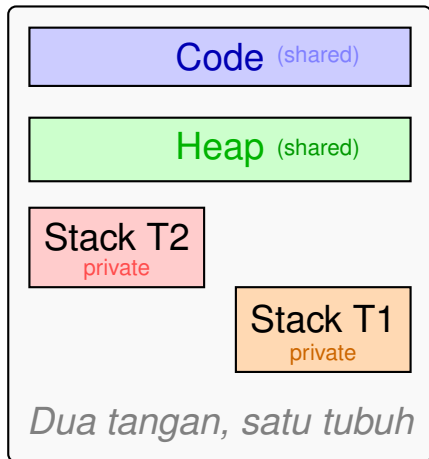
Tradeoff

Satu thread crash → **seluruh proses** ikut crash.

Shared vs Private dalam Thread

Semua thread berbagi satu address space — tapi stack-nya sendiri-sendiri

Process Address Space



Shared vs Private — Detail

Apa saja yang di-share dan apa yang private?

Di-shared antar thread (1 proses)

- Code segment
- Heap
- Global variables
- File descriptors
- Signal handlers

Private per thread

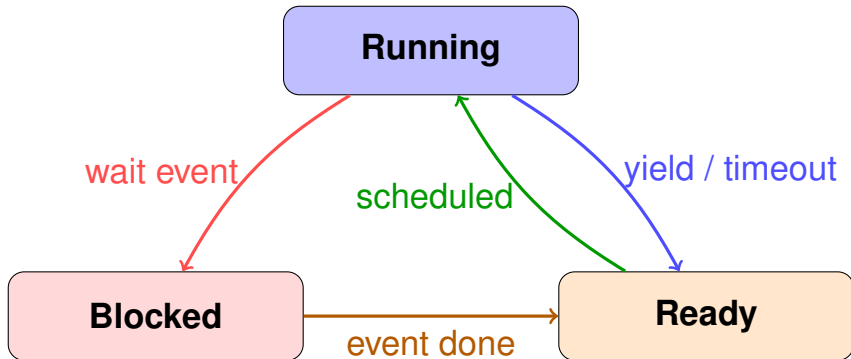
- Stack (variabel lokal)
- Register (termasuk PC)
- Thread ID
- errno

Mengapa ini penting?

Yang **di-share** bisa menyebabkan konflik jika diakses bersamaan. Yang **private** aman karena hanya dimiliki satu thread.

Thread Control Block & Thread States

Tiga keadaan thread dalam sistem operasi



Thread Control Block & Thread States — Detail

Bagaimana OS mengelola thread?

Thread Control Block (TCB):

- Analogus dengan PCB untuk process
- Berisi: register state, stack pointer, PC, thread ID, state

Thread States:

- **Running** — sedang dijalankan CPU
- **Ready** — siap, tapi CPU sedang sibuk
- **Blocked** — menunggu event (I/O, lock)

Context Switch antar Thread

1. Simpan register thread lama ke TCB
2. Muat register thread baru dari TCB
3. Stack pointer ganti → **stack berbeda**

Pthread API: Membuat dan Join Thread

Contoh sederhana dengan pthread

```
#include <stdio.h>
#include <pthread.h>

void *my_thread(void *arg) {
    char *name = (char *) arg;
    printf("Thread %s berjalan\n", name);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, my_thread, "A");
    pthread_create(&t2, NULL, my_thread, "B");
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Main selesai\n");
    return 0;
}
```

Penjelasan Kode

- `pthread_create()`: buat thread baru
- `pthread_join()`: tunggu thread selesai

Apakah output selalu berurutan?

Thread A → Thread B → Main?

Tidak! Urutan bisa berubah — ini disebut **non-determinism**.

Checkpoint 1

Cek pemahaman Bagian 1

1. Sebutkan tiga sumber daya yang di-**shared** antar thread dalam satu proses.
2. Sebutkan dua sumber daya yang **private** per thread.
3. Pada contoh kode `pthread_create`, apakah urutan output *Thread A* dan *Thread B* selalu sama? Mengapa?
4. Bagaimana OS melakukan *context switch* antar thread?

Kunci Jawaban Singkat

1. Shared: code segment, heap, file descriptors
2. Private: stack, register (termasuk PC)
3. Tidak — urutan ditentukan scheduler (*non-determinism*).
4. Simpan register ke TCB lama, muat dari TCB baru, ganti stack pointer.

Race Condition

Kode yang Tampak Benar

Dua thread, satu counter shared

```
#include <stdio.h>
#include <pthread.h>

int counter = 0; // shared!

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

3. Race Condition

Prediksi Awal

Dua thread masing-masing melakukan 1.000.000 increment.

counter = 2000000

Pertanyaan

Apakah hasilnya **selalu** 2.000.000?

Hasil Nyata: Tidak Stabil

Program yang sama, output bisa berbeda

Ekspektasi

2000000

Bayangkan...

Dua kasir menghitung stok barang di papan yang sama. Keduanya berniat benar, tapi kalau tulis-hapusnya bertabrakan, angka akhir bisa salah.

Beberapa hasil run

- `counter = 1234512`
- `counter = 1876023`
- `counter = 1543781`

Tidak selalu 2.000.000!

counter++ Bukan Satu Instruksi

Satu baris C bisa pecah jadi beberapa langkah mesin

Di kode C

```
counter++
```

Pesan utama

Satu baris C terlihat sederhana, tapi CPU melihatnya sebagai **tiga langkah terpisah**.

Di level assembly

```
MOV eax, [counter] ; LOAD nilai dari memori  
ADD eax, 1 ; ADD tambah 1 di register  
MOV [counter], eax ; STORE tulis balik ke memori
```

Langkah	Makna
LOAD	Baca nilai lama ke register
ADD	Tambah 1 di register
STORE	Tulis hasil ke memori

Interleaving yang Bermasalah

Masalah muncul saat dua thread saling menyela

Setup

Nilai awal `counter` = 50. Dua thread sama-sama menjalankan `counter++`.

Urutan kejadian

1. T1: LOAD $\rightarrow$ register = 50
2. OS *preempt* T1
3. T2: LOAD, ADD, STORE $\rightarrow$ `counter` = 51
4. T1 lanjut: ADD, STORE $\rightarrow$ tetap 51

Evolusi nilai

Momen	Nilai
Awal	50
T1 selesai LOAD	50
T2 selesai STORE	51
T1 selesai STORE	51

Inti: dua thread sama-sama membaca 50.

Lost Update

Hasil akhir salah walau tiap thread tampak benar

Seharusnya

$$50 + 1 + 1 = 52$$

Yang benar-benar terjadi

hasil akhir = 51

Analogi papan tulis

Bayangkan dua orang melihat angka 50 di papan tulis, lalu keduanya menghitung “50 + 1” di kepala masing-masing. Saat menulis balik, keduanya sama-sama menulis 51. Satu update hilang.

Inilah lost update.

Kapan Aman, Kapan Bahaya?

Tidak semua akses bersamaan otomatis salah

Aman

- Tiap thread bekerja pada **data private**
- Variabel lokal ada di **stack masing-masing**
- Thread tidak saling menimpa hasil

Bahaya

- Banyak thread menyentuh **data shared**
- Update terjadi hampir bersamaan
- Urutan eksekusi berubah-ubah

Aturan cepat

Kalau pertanyaannya menyangkut **siapa yang boleh menyentuh data yang sama pada waktu yang sama**, kita mulai masuk wilayah *race condition*.

Analogi Sederhana: Dua Kasir, Satu Catatan Stok

Mengapa hasil bisa salah padahal semua orang berniat benar?

Skenario

1. Stok awal tertulis **50**
2. Kasir A membaca angka itu
3. Sebelum A menulis balik, Kasir B juga membaca **50**
4. Keduanya menulis hasil "51"

Pelajarannya

Masalahnya bukan karena salah hitung.

Masalahnya karena **dua orang memakai catatan yang sama tanpa koordinasi**.

Hubungan ke thread

Kasir = thread, catatan stok = shared variable, tulis balik = STORE ke memori.

Definisi Race Condition

Kapan kita bilang sebuah program terkena race condition?

Definisi

Race condition adalah kondisi ketika hasil eksekusi program bergantung pada **timing** atau **urutan** eksekusi dua atau lebih thread yang mengakses data yang sama.

- **Non-deterministic**: kadang benar, kadang salah
- Sulit di-debug karena tidak selalu muncul
- Bergantung pada scheduler dan timing CPU
- Makin berbahaya saat beban sistem tinggi

Latihan: Race Condition atau Tidak?

Kuncinya: shared atau private?

Soal 1

```
void transfer(int amount) {  
    balance_A -= amount;  
    balance_B += amount;  
}
```

Jawaban

Ya, bisa race condition. balance_A dan balance_B adalah shared data.

Soal 2

```
void calculate(int x) {  
    int local = x * 2;  
    printf("%d\n", local);  
}
```

Jawaban

Tidak, jika local benar-benar variabel lokal. Ia berada di stack private milik tiap thread.

Checkpoint 2

Cek pemahaman Bagian 2

1. Mengapa `counter++` tidak aman jika dua thread mengakses `counter` yang sama?
2. Dalam contoh *interleaving*, mengapa hasil akhir bisa 51 padahal seharusnya 52?
3. Apa arti *non-determinism* pada program concurrent?

Kunci Jawaban Singkat

1. Karena `counter++` terdiri dari LOAD, ADD, dan STORE yang bisa disela thread lain.
2. Kedua thread membaca nilai lama yang sama, lalu salah satu update menimpa update lainnya.
3. Program yang sama bisa memberi hasil atau urutan berbeda karena timing eksekusi tidak tetap.

Critical Section

Apa Itu Critical Section?

Bagian kode yang tidak boleh dipakai bersamaan

Definisi

Critical section adalah bagian kode yang mengakses **shared resource** dan tidak boleh dieksekusi oleh lebih dari satu thread pada saat yang sama.

Contoh critical section

```
counter++; // shared variable
```

Di sini dua thread bisa menabrak data yang sama.

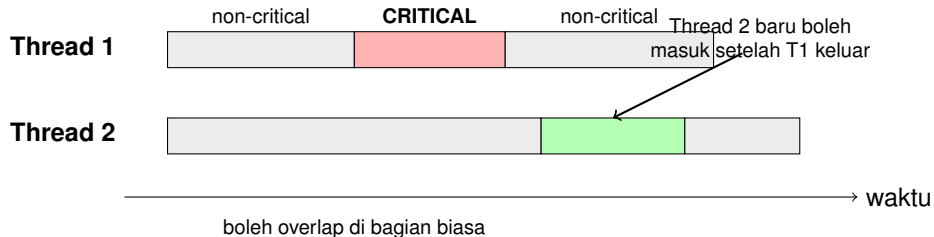
Bukan critical section

```
int temp = x * 2;  
printf("%d\n", temp);
```

temp adalah variabel lokal di stack private.

Visualisasi Critical Section

Yang tidak boleh overlap adalah bagian kritisnya



Analogi

Bayangkan hanya ada **satu kunci** untuk ruang arsip. Siapa pun yang sedang di dalam, orang lain harus menunggu di luar.

Tiga Syarat Solusi Critical Section

Solusi yang benar tidak cukup hanya “mengunci”

1. Mutual Exclusion

Hanya **satu thread** yang boleh ada di critical section pada satu waktu.

2. Progress

Kalau critical section kosong dan ada yang ingin masuk, harus ada keputusan masuk dalam waktu terbatas.

3. Bounded Waiting

Thread yang sudah meminta masuk tidak boleh menunggu tanpa batas.

Kalau Syaratnya Dilanggar, Apa yang Terjadi?

Tiap syarat mencegah masalah yang berbeda

Syarat	Pertanyaan kunci	Jika gagal
Mutual Exclusion	Apakah maksimum satu thread ada di CS?	<i>Race condition</i> tetap terjadi
Progress	Jika CS kosong, apakah ada yang bisa segera masuk?	Bisa macet atau saling tunggu
Bounded Waiting	Apakah ada batas menunggu?	Satu thread bisa kelaparan (<i>starvation</i>)

Cara mudah mengingat

ME mencegah tabrakan, **Progress** mencegah mandek, **Bounded Waiting** mencegah dianaktirikan.

Percobaan Naif 1: Flag Sederhana

Kelihatannya masuk akal, tapi bisa macet

```
int flag[2] = {0, 0};
```

```
// Thread 0  
flag[0] = 1;  
while (flag[1]);  
// critical section  
flag[0] = 0;
```

Masalah

Jika Thread 0 dan Thread 1 sama-sama set flag menjadi 1 hampir bersamaan, keduanya akan saling menunggu.

Akibat

Tidak ada yang maju. Ini melanggar **progress**.

Percobaan Naif 2: Turn Variable

Tidak tabrakan, tapi terlalu kaku

```
int turn = 0;

// Thread 0
while (turn != 0);
// critical section
turn = 1;
```

Masalah

Thread 1 tidak bisa masuk dua kali berturut-turut kalau Thread 0 belum “memberi giliran”, meski CS sedang kosong.

Akibat

Solusi ini juga gagal memenuhi **progress**.

Jadi, Kita Butuh Apa?

Teaser ke pertemuan berikutnya

Kesimpulan section ini

Kita butuh mekanisme yang:

- mencegah dua thread masuk bersamaan,
- tidak membuat semua thread macet,
- dan tetap adil untuk yang menunggu.

Preview minggu berikutnya

Di sinilah **lock** dan **mutex** masuk. Kita akan belajar bagaimana hardware dan OS membantu membuat critical section benar-benar aman.

Checkpoint 3

Cek pemahaman Bagian 3

1. Apa yang dimaksud dengan *critical section*?
2. Mengapa `counter++` termasuk critical section?
3. Apa beda *mutual exclusion*, *progress*, dan *bounded waiting*?
4. Mengapa solusi flag sederhana bisa membuat sistem macet?

Kunci Jawaban Singkat

1. Bagian kode yang mengakses data shared dan tidak boleh dieksekusi bersamaan.
2. Karena ia menyentuh variabel shared.
3. ME: cegah tabrakan; progress: cegah mandek; bounded waiting: cegah starvation.
4. Karena dua thread bisa saling menunggu.

Ringkasan & Penutup

Rangkuman: Tiga Ide Besar Hari Ini

Kalau pulang hanya bawa tiga hal, bawa tiga ini

Topik	Kata kunci	Inti
Thread	shared vs private	Banyak jalur eksekusi dalam satu proses, berbagi address space yang sama
Race Condition	timing matters	Hasil program bisa berubah karena urutan eksekusi thread tidak tetap
Critical Section	perlu proteksi	Bagian yang menyentuh data shared harus dijaga agar tidak dipakai bersamaan

Kalimat paling singkat

Thread membuat program lebih responsif, tapi **shared data** membuat kita harus hati-hati.

Checklist Keterampilan

Coba cek, mana yang sudah terasa paham?

- Menjelaskan beda *thread* dan *process*
- Menyebutkan mana yang *shared* dan mana yang *private*
- Membaca contoh kode `pthread`
- Menjelaskan kenapa `counter++` bisa salah
- Menggambar interleaving yang memicu *lost update*
- Mendefinisikan *race condition*
- Menjelaskan *critical section*
- Menyebutkan tiga syarat solusi yang benar

Kuis Penutup

Cek cepat sebelum kita akhiri

1. Sebutkan 3 hal yang di-*share* dan 2 hal yang *private* per thread.
2. Jika dua thread masing-masing menjalankan $x = x + 1$ pada variabel global x yang awalnya 0, nilai akhir x bisa berapa?
3. Jika hanya satu thread boleh masuk critical section, tetapi thread lain bisa menunggu selamanya, syarat mana yang dilanggar?

Kunci singkat

Shared: code, heap, global variable. Private: stack, register. Nilai akhir x bisa 1 atau 2. Jika ada yang menunggu selamanya, yang gagal adalah *bounded waiting*.

Ada Pertanyaan?

Minggu depan kita masuk ke cara nyata melindungi *critical section* dengan **lock** dan **mutex**.

“Programs must be written for people to read, and only incidentally for machines to execute.”
— Harold Abelson